DEEP LEARNING – UNIT I

B.Tech V Semester AI & ML | 23AM5T01 | Exam-oriented answers

UNIT I: Applied Math Foundations, Biological & McCulloch-Pitts Neuron, Perceptrons, PCA

PART A – SHORT ANSWERS (2 MARKS)

. Q1. What is Deep Learning? Differentiate it from Classical Machine Learning.

Deep Learning is a subset of Machine Learning that uses multi-layered artificial neural networks to learn complex patterns from large amounts of data.
Classical ML: Often uses shallow algorithms and requires manual feature extraction.
Deep Learning: Uses multi-layer neural networks and learns features automatically.

. Q2. Define Scalar, Vector, Matrix, and Tensor.

Scalar: A single numerical value (0D), e.g., 5.
Vector: A one-dimensional array, e.g., [1, 2, 3].
Matrix: A two-dimensional arrangement of numbers in rows and columns.
Tensor: A multi-dimensional array (3D or higher).

. Q3. Define Singular Matrix, Symmetric Matrix, and Skew-Symmetric Matrix.

Singular Matrix: A square matrix whose determinant is zero, det(A)=0.
Symmetric Matrix: A square matrix equal to its transpose, $A=A^T$.
Skew-Symmetric Matrix: A square matrix whose transpose equals its negative, $A^T=-A$.

. Q4. What are Eigenvalues and Eigenvectors? State their significance in Deep Learning.

Eigenvalues are scalar values that indicate the scaling factor of an eigenvector under a linear transformation.
Eigenvectors are non-zero vectors whose direction remains unchanged. They are used in PCA for dimensionality reduction and identifying important directions in data.

. Q5. What is PCA? What is its main objective?

Principal Component Analysis (PCA) is an unsupervised dimensionality-reduction technique. Its objectives are to reduce the number of features, preserve maximum variance, remove redundant information and improve computational efficiency.

. Q6. Explain One-Hot Encoding with an example.

One-Hot Encoding converts categorical data into binary vectors. For categories Red, Green and Blue: Red = [1,0,0], Green = [0,1,0], Blue = [0,0,1]. Each category has one 1 and the remaining values are 0.

. Q7. What is Image and Text Vectorization? Why is it mandatory before training neural networks?

Image Vectorization: Converts image pixels into numerical arrays or tensors.
Text Vectorization: Converts text into numerical representations such as vectors.
It is necessary because neural networks process numerical inputs.

. Q8. List the primary biological components of a Biological Neuron.

Dendrites: Receive signals.
Soma (cell body): Processes incoming signals.
Axon: Carries signals away from the cell body.
Synapse: Transfers signals between neurons.

. Q9. What are the core components of an Artificial Neuron?

The main components are inputs, weights, bias, summation unit and activation function. Its output is $y = f(\Sigma w_i x_i + b)$.

. Q10. Differentiate between Linearly Separable and Non-Linearly Separable datasets.

Linearly separable: Data points can be separated using a single straight line (e.g., AND and OR).
Non-linearly separable: Data points cannot be separated using a single straight line (e.g., XOR).

. Q11. State the significance of the Perceptron Convergence Theorem.

If a dataset is linearly separable, the Perceptron Learning Algorithm converges to a correct solution in a finite number of updates. It provides a guarantee for linear classification.

DEEP LEARNING – UNIT I

PART B | Expanded 10-Mark Exam Answers | B.Tech V Semester AI & ML

Unit topics: Applied Math Foundations, Biological & McCulloch-Pitts Neuron, Perceptrons and PCA.

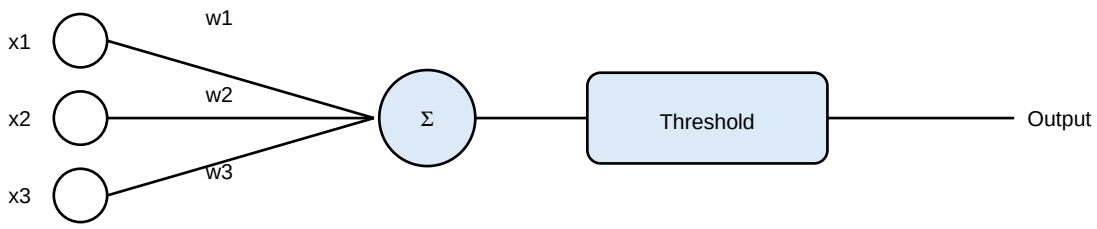
Writing tip: For a 10-mark answer, write the definition, explain each heading, include the relevant equation/table/diagram, and end with a short conclusion.

Q1. Explain the McCulloch-Pitts Neuron Model with a neat sketch. Explain its salient features, operational assumptions, and demonstrate Boolean AND, OR, and NOT gate implementations.

1. Introduction

The McCulloch-Pitts (M-P) neuron is one of the earliest mathematical models of an artificial neuron. It was proposed by Warren McCulloch and Walter Pitts in 1943. It represents a simplified biological neuron and uses binary inputs and binary output.

2. Structure of the M-P Neuron



The model consists of input signals, fixed weights, a summation unit, a threshold and a binary output. Each input is multiplied by its corresponding weight. The weighted inputs are added and compared with a threshold.

3. Mathematical Model

```
net = Σ(wi xi)
```

```
y = 1, if net ≥ θ; otherwise y = 0
```

Here, x■ is the binary input, w■ is its weight, θ is the threshold and y is the output.

4. Operational Assumptions

• Inputs and outputs are binary, i.e., either 0 or 1.

• The neuron receives signals through weighted connections.

• Weights are fixed and do not change during operation.

• The neuron fires only when the weighted sum reaches or exceeds the threshold.

• The output is determined by a step/threshold function.

• The model is a simplified representation and does not include biological learning or timing details.

5. Implementation of Boolean Gates

AND Gate: Choose w■=1, w■=1 and threshold θ=2. The output becomes 1 only when both inputs are 1.

X■	X■	Weighted sum	Output
0	0	0	0
0	1	1	0
1	0	1	0
1	1	2	1

OR Gate: Choose w■=1, w■=1 and threshold θ=1. The output becomes 1 when at least one input is 1.

X■	X■	Weighted sum	Output
0	0	0	0
0	1	1	1
1	0	1	1
1	1	2	1

NOT Gate: A single inhibitory input can implement NOT. When input is 0, output is 1; when input is 1, the inhibitory signal prevents firing.

X	Output
0	1
1	0

6. Salient Features

• Simple mathematical model of a neuron.

• Uses binary logic and threshold-based decision making.

• Can implement basic Boolean functions such as AND, OR and NOT.

• Provides the conceptual foundation for later artificial neural networks.

7. Limitations

• Cannot learn or adjust weights automatically.

• Uses only binary inputs and outputs.

• A single threshold unit cannot solve non-linearly separable functions such as XOR.

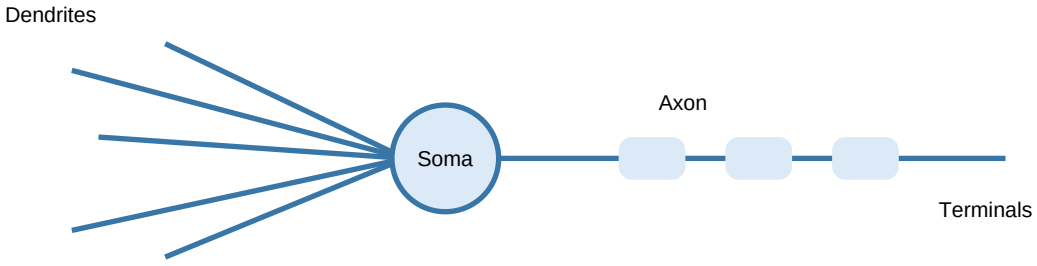
Conclusion: The McCulloch-Pitts neuron is a simple threshold-based computational model. Although limited, it established the basic idea of representing neuron activity mathematically.

Q2. Explain the structure and functioning of a Biological Neuron with a neat diagram. Discuss how the artificial computational unit was inspired by biological synaptic transmissions.

1. Introduction

A biological neuron is a specialized nerve cell and the basic information-processing unit of the nervous system. It receives information, processes it and transmits signals to other neurons through electrical and chemical processes.

2. Structure of a Biological Neuron



3. Main Parts and Functions

Part	Function
Dendrites	Receive signals from other neurons and carry them towards the cell body.
Soma (cell body)	Contains the nucleus and integrates/processes incoming signals.
Axon	Carries an electrical impulse away from the cell body.
Axon terminals	Form endings that pass signals to other cells.
Synapse	Junction between neurons where neurotransmitters transfer the signal.

4. Working of a Biological Neuron

• Reception: Dendrites receive chemical or electrical signals from neighbouring neurons.

• Integration: The soma combines the incoming signals.

• Threshold: If the membrane potential reaches the firing threshold, an action potential is generated.

• Transmission: The action potential travels along the axon.

• Communication: At the axon terminal, neurotransmitters are released across the synapse and influence the next neuron.

5. Inspiration for an Artificial Neuron

Biological component	Artificial neuron equivalent
Dendrites	Input values (x■, x■, …)
Synaptic strength	Weights (w■, w■, …)
Soma / cell body	Weighted summation unit
Firing threshold	Bias and activation threshold
Axon	Output of the artificial neuron

6. Artificial Neuron Equation

```
y = f(Σ wi xi + b)
```

The inputs are multiplied by weights, added with a bias, and passed through an activation function to produce an output.

7. Biological vs Artificial Neuron

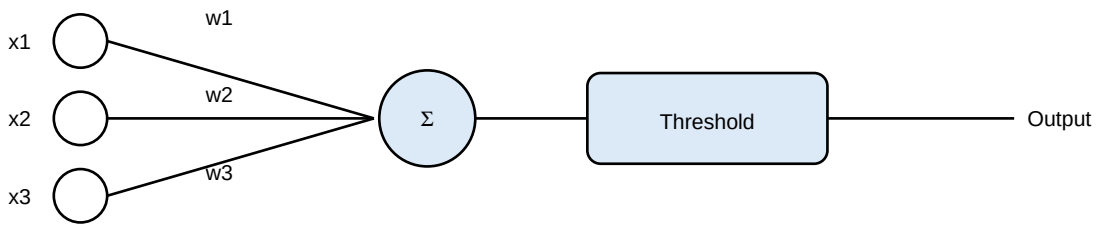
A biological neuron uses complex electrochemical processes, while an artificial neuron uses mathematical operations. The artificial model is inspired by the information flow and signal-strength concept of biological neurons, but it is a simplified abstraction.

Conclusion: Understanding biological neurons explains the inspiration behind artificial neural networks. Dendrites, synapses, soma and axons correspond conceptually to inputs, weights, summation and output.

Q3. What is a Linear Perceptron? Explain the Perceptron Learning Algorithm (PLA) and derive its step-by-step training rule and weight update mechanics.

1. Definition
A linear perceptron is a single-layer neural network used mainly for binary classification. It learns a linear decision boundary that separates input patterns belonging to two classes.

2. Architecture



The inputs are multiplied by their weights, summed with a bias, and passed through a threshold activation function.

3. Mathematical Model

```
z = Σ wi xi + b
```

```
y = 1, if z ≥ 0; otherwise y = 0
```

Here x■ represents input features, w■ represents weights, b is bias and y is the predicted class.

4. Perceptron Learning Algorithm

• Initialize all weights and bias to zero or small values.

• Select one training sample containing input vector x and target t.

• Calculate the net input z = Σw■x■ + b.

• Apply the step activation function to obtain predicted output y.

• Calculate the error e = t – y.

• Update weights and bias using the learning rule.

• Repeat for all training samples and continue for multiple epochs until there are no classification errors or a maximum number of epochs is reached.

5. Derivation of Weight Update Rule

The perceptron changes its weights according to the difference between the target output and predicted output. The error is:

$e = t - y$

The correction to each weight is proportional to the input and the error, scaled by the learning rate η:

$\Delta wi = \eta\ (t - y)\ xi$

Therefore, the new weight is:

$wi(new) = wi(old) + \eta\ (t - y)\ xi$

The bias is updated in the same way, treating its input as 1:

$b(new) = b(old) + \eta\ (t - y)$

Where η is the learning rate, t is the target, y is the predicted output and x■ is the input feature.

6. Interpretation of Weight Updates

• If t=y, the error is zero and no update is needed.

• If target is 1 but prediction is 0, weights are increased in the direction of the input.

• If target is 0 but prediction is 1, weights are adjusted in the opposite direction.

7. Advantages and Limitations

• Simple and computationally efficient.

• Suitable for binary classification.

• Converges for linearly separable training data.

• Cannot solve non-linearly separable problems such as XOR using a single layer.

Conclusion: The Perceptron Learning Algorithm trains a linear classifier by repeatedly correcting weights and bias based on classification error.

Q4. State and explain the Perceptron Convergence Theorem. Explain why a single-layer perceptron fails to solve the XOR classification problem with geometric reasoning.

1. Perceptron Convergence Theorem
The theorem states that if a set of training examples is linearly separable, the perceptron learning algorithm will find a separating weight vector after a finite number of updates, provided the standard perceptron update procedure is used.

2. Meaning of Linear Separability
A dataset is linearly separable if a single straight line (in two dimensions), plane (in three dimensions), or hyperplane (in higher dimensions) can separate the classes without classification errors.

3. Significance of the Theorem

• Provides a convergence guarantee for linearly separable data.

• Explains why the perceptron is suitable for simple binary classification tasks.

• Shows that training can finish after a finite number of corrections when a separating boundary exists.

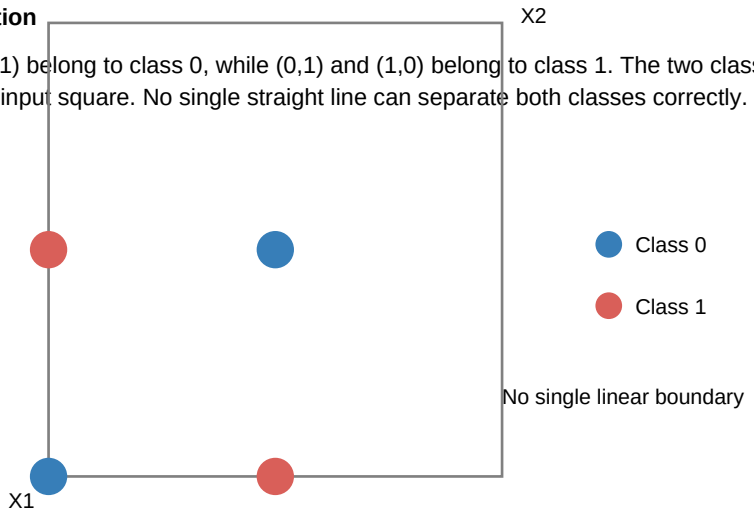
• Does not guarantee convergence for non-linearly separable data.

4. XOR Function and Truth Table

X■	X■	XOR output
0	0	0
0	1	1
1	0	1
1	1	0

5. Geometric Explanation

The points (0,0) and (1,1) belong to class 0, while (0,1) and (1,0) belong to class 1. The two classes occupy diagonally opposite corners of the input square. No single straight line can separate both classes correctly.



6. Why a Single-Layer Perceptron Fails

• A single-layer perceptron can form only one linear decision boundary.

• The XOR classes are arranged diagonally and cannot be separated by one line.

• Therefore, no choice of a single set of weights and bias can correctly classify all four XOR input combinations.

7. Solution
A Multi-Layer Perceptron (MLP) with at least one hidden layer and non-linear activation functions can combine multiple linear boundaries to represent the XOR function.

Conclusion: The convergence theorem guarantees learning only when the data is linearly separable. XOR is not linearly separable, so a single-layer perceptron cannot solve it.

Q5. Describe Threshold Logic using Perceptrons. Show how linearly separable logic functions are synthesized.

1. Introduction
Threshold logic is a method of implementing Boolean functions using a neuron that compares the weighted sum of its inputs with a fixed threshold. The output is binary.

2. Mathematical Representation

$$net = \Sigma\ wi\ xi$$

$$y = 1,\ if\ net \geq \theta;\ otherwise\ y = 0$$

The weights determine the contribution of each input, while the threshold determines when the neuron produces output 1.

3. Working Procedure

• Apply binary input values to the perceptron.

• Multiply each input by its corresponding weight.

• Add all weighted inputs to calculate the net input.

• Compare the net input with the threshold θ.

• If the net input is greater than or equal to θ, output 1; otherwise output 0.

4. Synthesis of Boolean Logic Gates

AND Gate: Set w■=1, w■=1 and θ=2. Only input (1,1) reaches the threshold.

X■	X■	Sum	AND output
0	0	0	0
0	1	1	0
1	0	1	0
1	1	2	1

OR Gate: Set w■=1, w■=1 and θ=1. Any input combination containing at least one 1 reaches the threshold.

X■	X■	Sum	OR output
0	0	0	0
0	1	1	1
1	0	1	1
1	1	2	1

NOT Gate: A single input can be inverted by using a negative weight and suitable threshold. For example, w=–1 and θ=0: output is 1 for x=0 and 0 for x=1.

Input x	Net = –x	NOT output
0	0	1
1	–1	0

5. Linear Separability
A Boolean function is linearly separable if its output classes can be separated by a single linear boundary in the input space. AND and OR are linearly separable; XOR is not linearly separable.

6. Applications

• Implementation of simple Boolean logic.

• Binary pattern classification.

• Threshold-based decision systems.

• Basic building block for neural network classifiers.

Conclusion: Perceptrons implement threshold logic by combining weighted inputs and a threshold. This provides a simple way to synthesize linearly separable Boolean functions.

Q6. Explain Principal Component Analysis (PCA) algorithm step-by-step: data centering, covariance matrix formulation, eigen-decomposition, and low-dimensional projection.

1. Definition
Principal Component Analysis (PCA) is an unsupervised statistical technique used for dimensionality reduction. It transforms a dataset with many possibly correlated features into a smaller set of uncorrelated variables called principal components, while retaining as much data variance as possible.

2. Objectives of PCA

• Reduce the number of input features.

• Preserve as much variance/information as possible.

• Remove redundancy caused by correlated features.

• Reduce storage and computational requirements.

• Support visualization of high-dimensional data.

3. Steps of PCA Algorithm

Step 1 – Arrange the Data: Represent the dataset as a matrix X with rows as observations and columns as features.

Step 2 – Data Centering: Calculate the mean of each feature and subtract it from every observation. This makes each feature have zero mean.

$$X_c = X - \mu$$

Here μ is the vector of feature means and X_c is the centered data matrix.

Step 3 – Covariance Matrix: Calculate the covariance matrix to measure how features vary together.

$$C = (1/(n-1))\ X_c^T X_c$$

The diagonal elements represent feature variances; off-diagonal elements represent covariances between pairs of features.

Step 4 – Eigen Decomposition: Find eigenvalues and eigenvectors of the covariance matrix.

$$C\ v = \lambda\ v$$

Eigenvectors give the directions of principal components. Eigenvalues indicate the amount of variance along those directions.

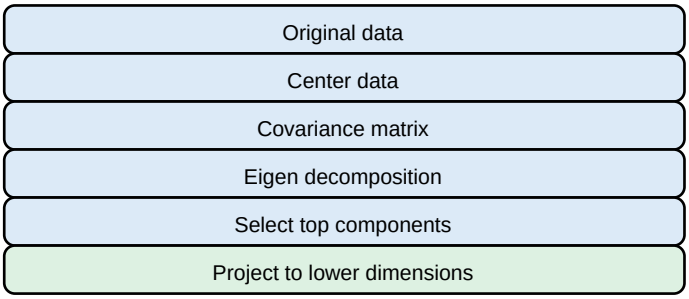
Step 5 – Sort and Select Components: Sort eigenvalues in descending order and select the eigenvectors corresponding to the largest eigenvalues. These directions retain the greatest variance.

Step 6 – Projection: Form a projection matrix W using the selected eigenvectors and transform the centered data.

$$Z = X_c\ W$$

Z is the lower-dimensional representation of the original data.

4. PCA Flow Diagram



5. Advantages

• Reduces the number of dimensions.

• Removes correlated/redundant information by creating uncorrelated principal components.

• Can reduce computation and storage.

• Can help visualize data in two or three dimensions.

6. Limitations

• PCA is a linear dimensionality-reduction method.

• Principal components may be difficult to interpret.

• Features should often be scaled when they have very different units or ranges.

Conclusion: PCA centers the data, computes its covariance matrix, finds eigenvectors and eigenvalues, selects the most important components and projects the data into a lower-dimensional space.

Q7. Discuss various types of matrices in linear algebra (Diagonal, Orthogonal, Identity, Inverse, Singular) and explain their importance in deep neural networks.

1. Introduction
A matrix is a rectangular arrangement of numbers in rows and columns. In deep learning, matrices represent datasets, weights, transformations and intermediate computations.

2. Types of Matrices

a) Diagonal Matrix: A square matrix in which all non-diagonal elements are zero.

```
A = [[2,0,0], [0,3,0], [0,0,4]]
```

It simplifies multiplication and many matrix operations.

b) Orthogonal Matrix: A square matrix whose transpose is equal to its inverse.

```
A■ A = A A■ = I
```

Its rows and columns are orthonormal. Orthogonal transformations preserve vector lengths and angles.

c) Identity Matrix: A square matrix with 1s on the main diagonal and 0s elsewhere.

```
I = [[1,0], [0,1]]
```

For a compatible matrix A, AI = IA = A.

d) Inverse Matrix: For a non-singular square matrix A, its inverse A■¹ satisfies:

```
A A■¹ = A■¹ A = I
```

The inverse exists only when det(A) is non-zero.

e) Singular Matrix: A square matrix is singular if its determinant is zero.

```
det(A) = 0
```

A singular matrix has no ordinary inverse.

3. Summary Table

Matrix type	Definition / property	Key point
Diagonal	All off-diagonal elements are zero	Simplifies operations
Orthogonal	A■A = I	Preserves lengths and angles
Identity	Diagonal entries 1; others 0	Multiplicative identity
Inverse	AA■¹ = I	Exists for non-singular square matrices
Singular	det(A)=0	No inverse

4. Importance in Deep Neural Networks

• Weight matrices: Neural network layers use matrices to represent trainable weights and perform linear transformations.

• Orthogonal transformations: Orthogonal weights can help preserve signal magnitudes and support stable propagation in some architectures.

• Identity matrices: Used in matrix algebra and as identity transformations; they also appear in residual/skip connections conceptually.

• Inverse matrices: Useful in solving linear systems and in some mathematical derivations, although neural network training rarely requires explicitly computing large matrix inverses.

• Singular matrices: Understanding singularity helps explain rank deficiency and why some linear systems do not have unique solutions.

Conclusion: Matrix types provide the mathematical foundation for representing and transforming data in neural networks. Their properties are useful for understanding computations, stability and linear algebra operations.

Q8. Explain Eigenvalues and Eigenvectors with a numerical example and describe their role in dimensionality reduction and feature decorrelation.

1. Introduction
Eigenvalues and eigenvectors are important linear algebra concepts used to understand the behavior of square matrices under linear transformations.

2. Definitions

• Eigenvector: A non-zero vector whose direction does not change when a matrix transformation is applied; only its magnitude may change.

• Eigenvalue: The scalar factor by which the corresponding eigenvector is scaled.

3. Mathematical Equation

$$A v = \lambda v$$

A is a square matrix, v is an eigenvector and λ is its eigenvalue. Eigenvalues are found by solving:

$$\det(A - \lambda I) = 0$$

4. Numerical Example
Consider:

$$A = [[2,0], [0,3]]$$

The characteristic equation is:

$$|A - \lambda I| = 0$$

$$(2 - \lambda)(3 - \lambda) = 0$$

Therefore, the eigenvalues are $\lambda_1=2$ and $\lambda_2=3$.

For $\lambda_1=2$, an eigenvector is:

$$v_1 = [1,0]$$

For $\lambda_2=3$, an eigenvector is:

$$v_2 = [0,1]$$

Thus, the matrix scales the first coordinate direction by 2 and the second coordinate direction by 3.

5. Role in Dimensionality Reduction

• In PCA, eigenvectors of the covariance matrix give the principal component directions.

• Eigenvalues indicate the variance captured along each principal component.

• Components with the largest eigenvalues are selected to retain the most important variation.

• Components with small eigenvalues may be discarded to reduce the number of dimensions.

6. Role in Feature Decorrelation
When PCA projects centered data onto the eigenvectors of its covariance matrix, the covariance matrix in the principal-component coordinate system becomes diagonal (for the standard symmetric covariance matrix). Thus, different principal components are uncorrelated.

7. Applications

• PCA and dimensionality reduction.

• Data compression and visualization.

• Analysis of covariance and feature relationships.

• Understanding directions of variation in datasets.

Conclusion: Eigenvalues measure the amount of scaling or variance, while eigenvectors give the corresponding directions. They are central to PCA and feature decorrelation.

Q9. Explain Vectorization techniques for unstructured data: Image Vectorization (pixel matrices, channel tensors) and Text Vectorization (Bag of Words, TF-IDF, Word Embeddings).

1. Introduction
Vectorization is the process of converting unstructured information such as images and text into numerical arrays or vectors. Neural networks require numerical input, so vectorization is an important preprocessing step.

2. Image Vectorization

An image consists of pixels. Each pixel is represented by numerical intensity values.

• Grayscale image: Represented as a two-dimensional matrix in which each element represents pixel intensity.

• RGB image: Represented as a three-dimensional tensor with Red, Green and Blue channels.

• Pixel values are commonly normalized to a range such as 0–1 before being supplied to a neural network.

Example: A grayscale image of height H and width W has shape H×W. An RGB image has shape H×W×3.

3. Text Vectorization
Text must be converted into numbers because a neural network cannot directly process raw words.

a) Bag of Words (BoW)

• Creates a vocabulary of unique words from the documents.

• Represents each document using word counts or frequencies.

• It is simple, but ignores grammar and word order.

Document	I	like	AI	ML
I like AI	1	1	1	0
I like ML	1	1	0	1

b) TF-IDF (Term Frequency–Inverse Document Frequency)

TF-IDF assigns a weight to a word based on its frequency in a document and how rare it is across the document collection. Common words receive lower importance, while words that are more distinctive to a document can receive higher weights.

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t)$$

c) Word Embeddings

• Represent words as dense, low-dimensional numerical vectors.

• Capture semantic and contextual relationships better than simple word counts.

• Words with related meanings can have similar vector representations.

• Examples include Word2Vec and GloVe.

4. Comparison of Text Vectorization Methods

Method	Representation	Main characteristic
Bag of Words	Sparse word-count vector	Ignores word order
TF-IDF	Sparse weighted vector	Down-weights common words
Word Embeddings	Dense vector	Captures semantic similarity

5. Why Vectorization is Important

• Converts raw images and text into numerical form.

• Makes unstructured data suitable for neural network input.

• Provides a consistent representation for training.

• Supports feature extraction and pattern learning.

6. Limitations
Bag of Words and TF-IDF do not naturally capture word order or deep semantic meaning. Word embeddings improve semantic representation but may still require context-aware models for words with multiple meanings.

Conclusion: Image vectorization represents pixels as matrices or channel tensors, while text vectorization uses methods such as BoW, TF-IDF and word embeddings. These methods make unstructured data suitable for deep learning.

UNIT I – PART B COMPLETED

All 9 Part-B questions have been expanded into 10-mark style answers with definitions, subheadings, equations, examples, tables and diagrams wherever useful.

Exam presentation: Underline key terms, draw the supplied diagrams neatly, write equations on separate lines and include the conclusion.